ASSIGNMENT REPORT

FastSpeech2 TTS for English and Telugu

Name: G.V. Akash | **Date:** 7 November 2026

1. Objective

The objective of this assignment is to implement and evaluate the FastSpeech 2 text-to-speech (TTS) model separately for English and Telugu, referring to the original research paper and an available GitHub implementation.

2. Summary and Architecture

This project presents two separately trained TTS models, one for English and one for Telugu, built on top of the reference implementation (github.com/ming024/FastSpeech2) and the core architecture of the FastSpeech 2 paper (Ren et al., 2020, arXiv:2006.04558). FastSpeech 2 is the successor of FastSpeech, which uses a teacher–student distillation pipeline; that pipeline suffers from information loss due to data simplification and has limitations with one-to-many mapping. The FastSpeech 2 pipeline works as follows

1. The input text is converted into phonemes (tokens).
2. The phoneme embeddings, together with positional encodings, form a sequence of vectors that is fed into the encoder.
3. The FFT (Feed-Forward Transformer) encoder applies self-attention, thanks to the Transformer architecture, and processes the vectors through 1D convolutions to produce a contextual hidden representation.
4. These hidden states are fed into FastSpeech 2's variance adaptor, which predicts three quantities for each phoneme its duration in frames, its pitch (fundamental frequency, F0) and its energy. The training targets for pitch come from an F0 tracker (WORLD).
5. The length regulator uses the durations (ground-truth durations from MFA during training, predicted durations at inference), which allows the model to work non-autoregressively. It repeats each phoneme's vector for as many mel frames as its predicted duration, forming a longer sequence of frame vectors.
6. The decoder turns these into acoustic features using FFT blocks, producing a mel spectrogram.
7. The mel spectrogram is fed to a HiFi-GAN vocoder, which converts it into a waveform.

2.1 Constraints and scope

I do not have a dedicated GPU. All GPU work ran in Kaggle notebooks on a T4, within Kaggle's free GPU quota of 30 hours and its 12-hour session limit. Each language therefore got one training run with a 7-hour budget, and there was no budget for hyper-parameter searches or repeated runs.

What was not done: AI4Bharat IndicMFA, a set of pretrained Telugu aligners that I found only after training, was not used. Instead, an MFA acoustic model was trained on the same ~5 h of selected Telugu speech. IndicMFA provides pretrained letter-level Telugu aligners, which would probably give better phone boundaries. Using them would require a letter-based input representation instead of the phone set used here.

Evaluation of the Telugu TTS: I am not a native Telugu speaker. The Telugu audio was judged by two listeners (21M and 45F) who were given no context about the project. There was no MOS test.

3. Implementation approach

In short, I downloaded both datasets from Hugging Face using a read token, selected ~5 h for training and ~1 h for validation, and wrote them out as 22.05 kHz WAV files with normalized transcripts. data_prep.build_lexicon builds the pronunciation dictionary for MFA from the language front-end. align.py uses the pretrained MFA model without modification for English, while a separate MFA model was trained for Telugu; its settings are in mfa_train_config.yaml.

train.py and trainer.py run the FastSpeech 2 training with a 7-hour wall-clock budget, CSV loss logs and best-checkpoint tracking.

Both models use the pretrained HiFi-GAN universal vocoder from the reference repository, which was not fine-tuned. Each model was trained once, for 7 hours, on a free Kaggle T4 GPU. The main additions to the reference code are a new English text front-end, a rule-based Telugu G2P with number normalization, a Telugu aligner trained from scratch with the Montreal Forced Aligner (MFA), and a fixed data-selection and split procedure.

3.1 Components reused from the reference implementation

Component	Configuration used (both languages)
Encoder	4 FFT blocks, hidden 256, 2 heads, conv filter 1024, kernels [9, 1], dropout 0.2
Decoder	6 FFT blocks, hidden 256, 2 heads, dropout 0.2
Variance adaptor	Duration, pitch and energy predictors (2 conv layers, filter 256, kernel 3, dropout 0.5); length regulator; pitch and energy quantized into 256 linear bins and embedded
Pitch and energy targets	Phone-level (one value per phone), z-score normalized
PostNet	5 one-dimensional convolutions, 512 channels, kernel 5
Speaker embedding	One table of size 256, added to the encoder output; turned on automatically when more than one speaker survives preprocessing (multi_speaker: auto)
Loss	L1 on the mel before and after the PostNet, plus MSE on pitch, energy and log-duration, unweighted sum
Optimiser	Adam ($\beta = 0.9, 0.98$; $\varepsilon = 1e\text{-}9$), Noam schedule with 4,000 warm-up steps, gradient clipping at 1.0
Vocoder	HiFi-GAN generator, universal checkpoint (fastspeech2_core/hifigan/generator_universal.pth.tar.zip), 22.05 kHz, hop 256
Size	35.46 M parameters (English) and 35.43 M (Telugu), counted from the exported checkpoints. The models differ only in the size of the speaker table (114 × 256 vs 2 × 256).

3.2 Changes shared by both languages

File	Change	Reason
preprocessor.py	MFA ≥ 2 writes silences as **empty** intervals, which tgt drops by default. They are now read and mapped to sp, and consecutive silences are merged.	MFA sometimes represents silence as an empty , which the original code ignored. Without this fix, the timing no longer matches the mel spectrogram, and after a pause the duration, pitch and energy values become misaligned.
preprocessor.py	Pitch and energy statistics are computed **on the training set only**. Utterances are dropped for spn, a non-silence phone longer than 100 frames, or more than 1,550 frames.	Keeps validation data out of the normalization statistics and removes utterances that were aligned badly.
model/modules.py	The energy predictor used p_control instead of e_control.	A bug in the reference implementation made the energy predictor use the pitch control value instead of the energy control value. This was corrected, so pitch and energy can now be controlled independently.
audio/stft.py, audio/audio_processing.py	Removed the hard-coded .cuda(); librosa calls now use keyword arguments.	The code crashed on CPU and in multiprocessing; the librosa ≥ 0.10 API requires keyword arguments.
trainer.py	One config file, --max_hours budget, resume from the latest checkpoint, CSV loss logs, best checkpoint by validation loss (best.json), checkpoint pruning, single-GPU DataParallel.	Fits the Kaggle session limits and provides the loss plots.
synthesis.py	Rewritten. Writes the test-set audio, plots per-phone duration, F0 and energy, measures F0 and energy on the generated audio, and exports CSVs.	The inference and evaluation pipeline was rewritten to generate the required test audio samples and variance-adaptor plots for duration, pitch (F0) and energy. The results are also exported as CSV files for further analysis.
config.py	One config.yaml per language, split back into ming024's three config dictionaries.	Separate configuration files are maintained for English and Telugu. Each is converted into the format expected by the original FastSpeech 2 implementation.

3.3 Changes made for English

Text normalizer (text/english_frontend.py). It handles times (10:45 AM as ten fortyfive A M), dates, currency, etc. The reference code's g2p_en produced ten fortyfive for 10:45 and read AM as the verb am.

MFA dictionary. For every word, the MFA dictionary contains all CMUdict variants, both heteronym readings, letter names, and g2p_en predictions for out-of-vocabulary words. It has 6,421 words and 7,688 pronunciations (English/data_meta/lexicon.txt), so MFA can choose the variant that was actually spoken.

Alignment uses the pretrained english_us_arpa MFA model (ARPAbet with stress, the same phone set as g2p_en) via mfa align.

Speakers. Svarah has no speaker-ID column. The tuple of speaker metadata (gender, age group, L1, state, district, education, job, domain) is hashed into a pseudo-speaker ID, which gave **114** IDs (English/data_meta/speakers.json).

3.4 Changes made for Telugu

Telugu G2P (text/telugu_frontend.py). The reference code supports only English and Mandarin. Telugu spelling is close to phonemic, so a rule-based converter is used. The inventory has **48 phones**, written in lowercase ASCII so that they never collide with ARPAbet in the shared symbol table. Two of them (f, z) never occur in the training data (Telugu/data_meta/preprocess_summary.json, phone_counts_train).

Telugu number normalization. Telugu digits are mapped to ASCII; cardinals use the Indian system (2026 as రెండు వేల ఇరవై ఆరు); ordinals use -వ (12వ as పన్నెండవ); clock times are expanded (10:45 as పది నలభై ఐదు); decimals and percentages are also handled.

Training our own MFA aligner. MFA is a small speech recognizer of its own (a Kaldi GMM-HMM). It has to learn what each Telugu phoneme sounds like before it can mark where each phoneme starts and ends. English does not need this because MFA ships the pretrained english_us_arpa model. No Telugu model was available, so one was trained on the selected Telugu clips during the align stage of Telugu/preprocess.py. This ran on the CPU before any GPU training.

4. Data and training setup

Only clips between 1 and 15 seconds long were kept clips shorter than 1 s carry too little information, and clips longer than 15 s make training compute-heavy.

For the English dataset, which was not recorded in a studio, I also removed the noisiest 10% of clips (513 clips) using an SNR proxy. The Telugu dataset was recorded in a studio, so it did not need this step.

The validation set was prepared first, so that all ~113 distinct English speakers get a chance to appear in it before the training set is filled; this avoided a shortage of speakers in the validation set. The selection code is identical for both languages (data_prep.select_and_split, seed 1234). Clips are grouped by normalized transcript, so the training and validation sets **do not** overlap.

Every selected clip is resampled to 22.05 kHz, converted to mono and peak-normalized to 0.95. Because the female Telugu voice and all of Svarah were recorded at 16 kHz, their audio has no content above 8 kHz. This matches the mel range used (0–8 kHz).

4.1 Training configuration

Setting	Value
Batch size	16
Learning rate	Noam schedule; the planned annealing (50k–90k steps) was never reached
Precision	fp32
Validation	Whole validation set every 1,000 steps
Checkpoints	Every 5,000 steps and at each new best validation loss

Setting	Value
Budget	7 h wall clock, Kaggle T4 GPU
Steps reached	English 34,898 (~1.38 steps/s, ~177 passes over the training set); Telugu 30,980 (~1.23 steps/s, ~229 passes)
Checkpoint for inference	Lowest validation total loss: English step 31,000, Telugu step 11,000

5. Training and validation loss

Loss	English				Telugu			
	Val at 1k	Best val (step)	Val at end	Train at end	Val at 1k	Best val (step)	Val at end	Train at end
Total	2.811	2.498 (31k)	2.523	0.886	2.292	1.971 (11k)	1.972	0.728
Mel (post-net)	0.978	0.746 (11k)	0.759	0.414	0.748	0.602 (10k)	0.606	0.341
Pitch	0.391	0.391 (1k)	0.418	0.020	0.521	0.504 (26k)	0.504	0.021
Energy	0.368	0.359 (21k)	0.370	0.024	0.220	0.198 (16k)	0.199	0.018
Log-duration	0.211	0.210 (2k)	0.218	0.014	0.066	0.054 (7k)	0.057	0.008

5.1 Overfitting

Both the English and Telugu models overfit, particularly in the variance predictors. The mel decoder continued to perform relatively well when given ground-truth prosodic information, but the duration, pitch and energy predictors showed a larger gap between training and validation performance. Their best validation values come early: at **step 1,000** for English (pitch) and **step 7,000** for Telugu (duration). The best mel loss values are at 10k–11k steps in both languages. The English validation losses are higher than the Telugu ones for every component the English model learns 114 voices with a median of 1.9 minutes each, while the Telugu model has two studio voices with about 2 hours each. The validation mel loss is computed with ground-truth durations, pitch and energy (teacher forcing), as in the reference code.

Both languages showed a temporary increase in one validation loss around step 3,000, which recovered within the next few thousand steps.

6. Speech evaluation

6.1 English

ID	Score	Pitch	Energy	Duration (timing)	Clarity and pronunciation
E1	2 / 5	Perfect	Good	Very poor	Robotic; “antidisestablishmentarianism” is not clear.

ID	Score	Pitch	Energy	Duration (timing)	Clarity and pronunciation
E2a	2.5 / 5	Perfect	Average	Good	The first words are unclear.
E2b	3 / 5	Poor the last word, “method”, ends on a high note, so the statement sounds like a question.	Average	Good	Best pronunciation, but still sounds robotic.
E3	2 / 5	Problems on the last word, “unexpectedly”.	Good	Acceptable	“rhyth m” is pronounced “raitham”.
E4	3 / 5	Best in the set	Best in the set	Best in the set	Lacks clarity. The Telugu word “andaroo” sounds as if a speaker from North or Northeast India is saying it.

E2a and E2b audibly distinguish conduct as a noun from conduct as a verb.

6.2 Telugu

I am not a native Telugu speaker. The generated Telugu audio was played to a classmate and his mother without telling them what it was or where it came from.

I could still judge the pitch, it was perfect in all of the samples, and every sentence ended on a lower pitch, as expected.

Listener	Comments
Classmate, age 21	It sounds like AI robotic and very fast-paced.
Classmate’s mother, age 54	Understood the Telugu clearly. The pronunciation is correct, but the pacing is not at a human level. With this audio one can clearly understand what is being said, but not in the way humans speak.

7. Variance-adaptor analysis

7.1 Per-sentence analysis

E1

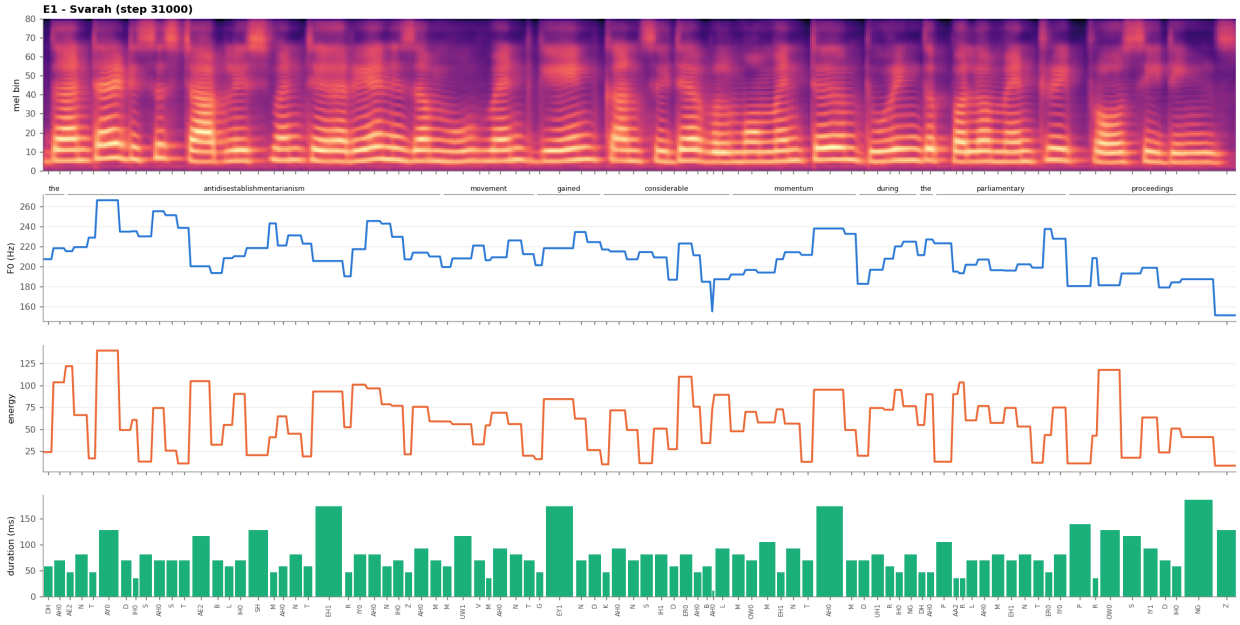


Figure 1. E1: mel spectrogram, predicted F0, energy and duration per phone.

Duration: The predicted speech timing is generally steady, averaging approximately 72 ms per phoneme. Stressed vowels receive longer durations, while unstressed sounds such as the schwa in “ble” are much shorter. Some phonemes near the end of “proceedings” receive relatively long durations, suggesting a stretched ending.

Pitch (F0): The predicted pitch starts near 210 Hz, reaches a peak of approximately 266 Hz, and generally varies between 190 and 255 Hz across the sentence. Small pitch increases occur around stressed syllables, followed by a fall to approximately 151 Hz at the end.

Energy: Vowels generally have higher predicted energy, while consonants such as S and T have lower values. The predicted and measured energy have a correlation of 0.92, indicating strong agreement between the model’s energy prediction and the generated audio.

E2a vs E2b

conduct lasts 534 ms in both sentences.

	1st vowel	2nd vowel
Noun (E2a): duration	AA1 93 ms	AH0 70 ms
Verb (E2b): duration	AA0 46 ms	AH1 58 ms
Noun (E2a): F0	216 Hz	207 Hz
Verb (E2b): F0	251 Hz	200 Hz

	1st vowel	2nd vowel
Noun (E2a): energy	87	91
Verb (E2b): energy	78	90

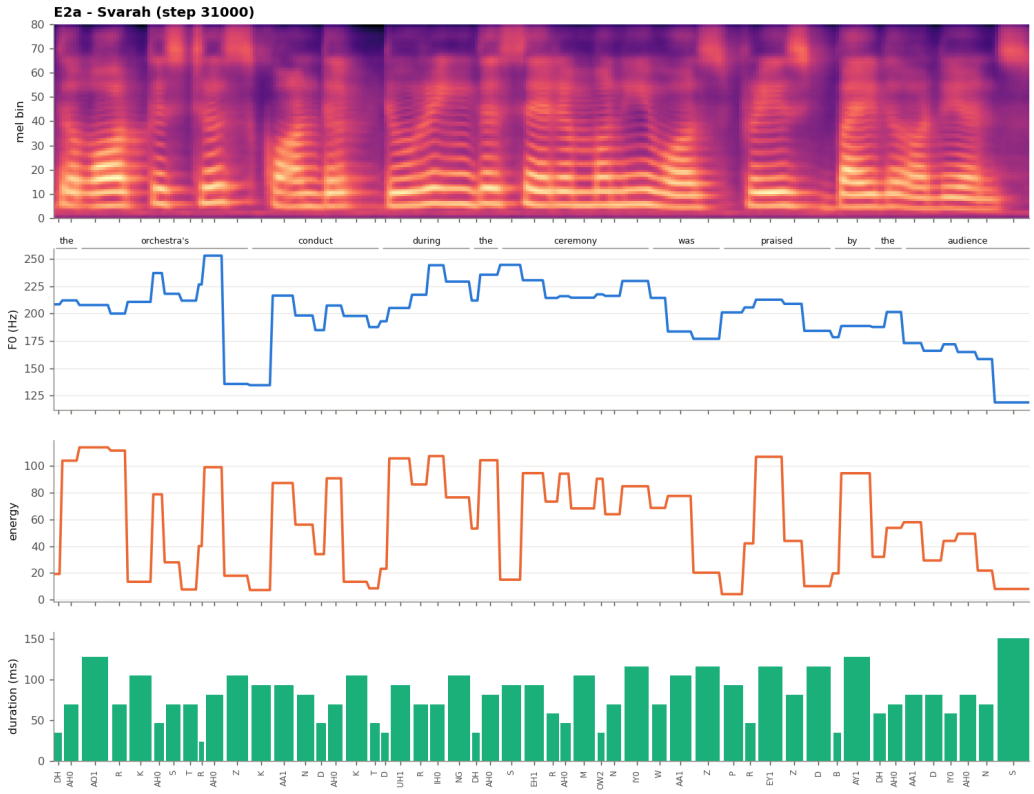


Figure 2. E2a: mel spectrogram, predicted F0, energy and duration per phone.

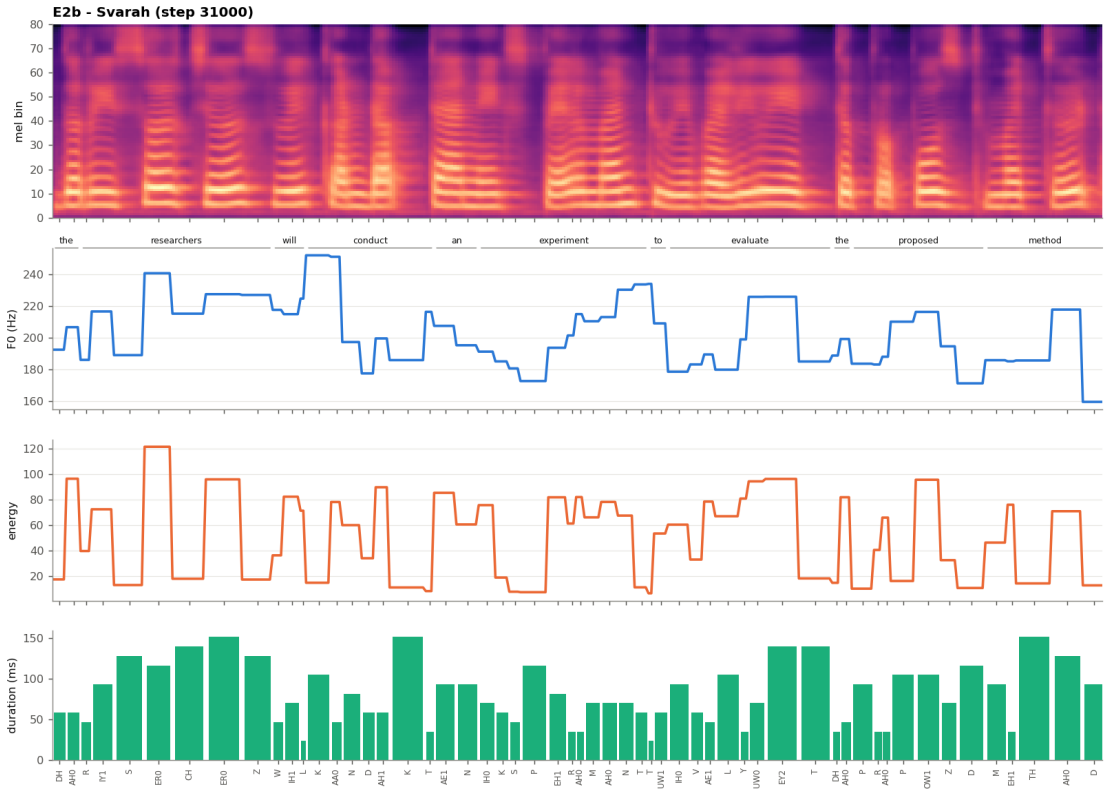


Figure 3. E2b: mel spectrogram, predicted F0, energy and duration per phone.

Duration: Duration provides the clearest indication of lexical stress in these examples, with the stressed vowel receiving a relatively longer duration.

Pitch (F0): The pitch pattern does not directly follow lexical stress. In the verb form, the F0 peak occurs on the unstressed first syllable, suggesting that pitch is influenced more by the overall intonation of the sentence, particularly the pitch movement following “will”.

Energy: Energy shows only a small difference between the two forms and therefore does not strongly distinguish the stress pattern.

Sentence-level prosody: E2a ends with a falling pitch, down to 119 Hz on “audience”. E2b does not end in a clear fall: the last vowel of “method” rises from about 185 Hz to about 218 Hz, which matches the question-like ending heard in E2b also contains relatively long fricative sounds, including 10–13 frames per phoneme in “researchers” and a 151 ms TH sound in “method”.

E3

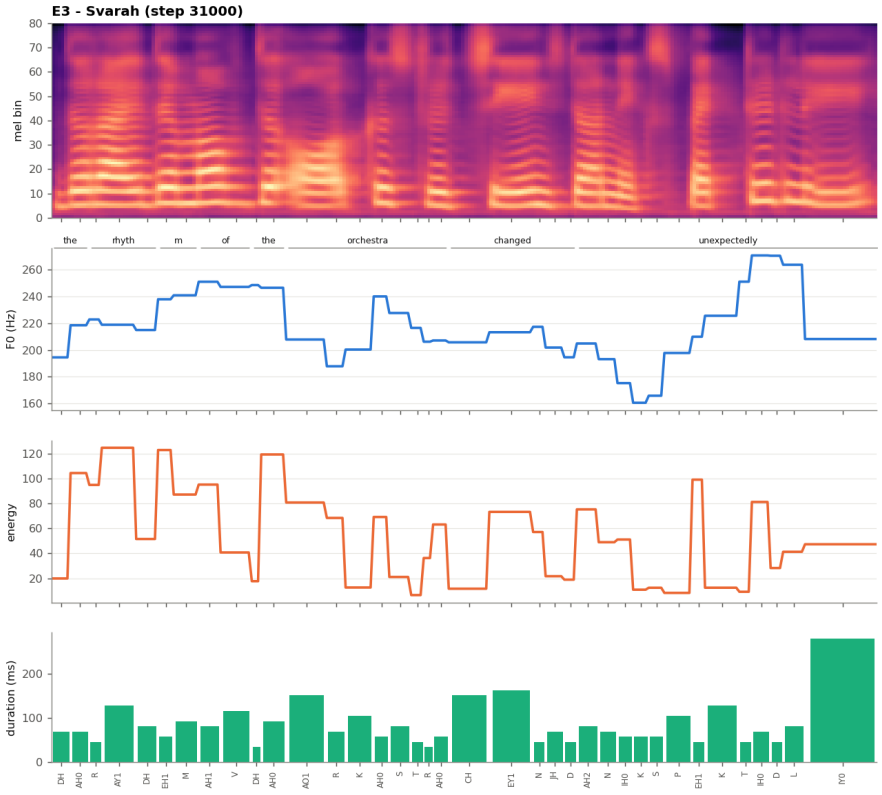


Figure 4. E3: mel spectrogram, predicted F0, energy and duration per phone.

Duration: The sounds in “rhyth m” run directly into each other without a pause. The final IY0 vowel in “unexpectedly” has the longest duration among the final vowels in the English test set, lasting approximately 279 ms.

Pitch (F0): The sentence shows a rise–fall pitch pattern rather than a simple falling contour. F0 rises from approximately 220 Hz to 251 Hz, falls around “orchestra”, rises again to approximately 270 Hz, and finally ends near 208 Hz. The predicted and measured pitch differ considerably in some regions, particularly around “orchestra” and the unvoiced consonants in “unexpectedly”. The F0 correlation of 0.69 is the lowest among the English test sentences.

Energy: The energy pattern is consistent, with a strong correlation of 0.96 between the predicted energy and the energy measured from the generated audio.

E4

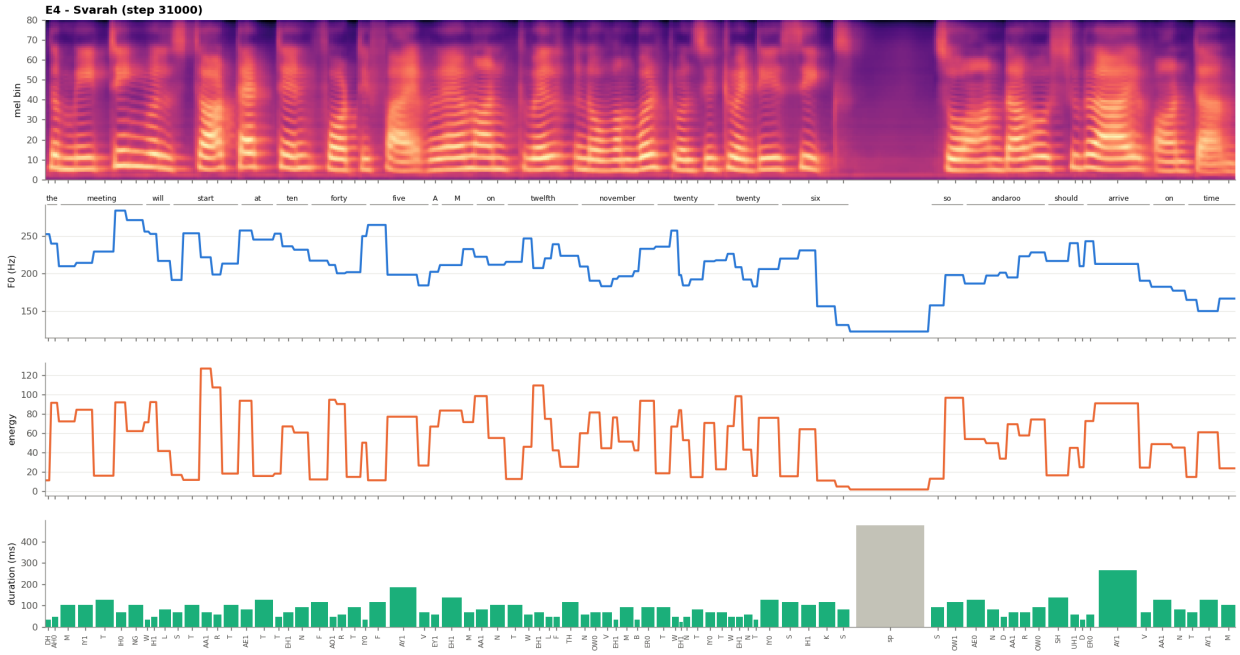


Figure 5. E4: mel spectrogram, predicted F0, energy and duration per phone.

Duration: The comma pause is the longest unit in the sentence, lasting approximately 476 ms. Stressed vowels such as AY1 in “arrive” and “five” also receive relatively long durations, while the spoken letter “A” is much shorter.

Pitch (F0): The pitch starts high (about 253–284 Hz on “the meeting”), drifts down through the date and falls sharply into the pause after the comma, with the last vowel before it (“six”) at about 231 Hz. After the pause it resets upwards (157 as 198 Hz on “so”) before falling again towards the end of the sentence. This shows a clear phrase-level intonation pattern.

Energy: Energy is close to zero during the pause and reaches a clear peak on the AA1 vowel in “start”, showing a strong contrast between silence and voiced speech.

8. English vs Telugu

8.1 Comparison

	English	Telugu
Data	114 pseudo-speakers, median 1.9 min each, 16 kHz	2 studio voices, about 2 h each, 16 / 48 kHz
Front-end	CMUdict + g2p_en with sentence-level POS; normalizer for times and dates	Rule-based G2P, no exception list; Telugu number expansion
Aligner	Pretrained english_us_arpa	Trained from scratch on the same 5 h
Best validation total / mel / duration loss	2.498 / 0.746 / 0.210	1.971 / 0.602 / 0.054
Validation plateau	About 7–11k steps	About 7–11k steps

	English	Telugu
Synthesized speaking rate	11.4–13.0 phones/s	12.1–14.1 phones/s
Split-word test	“rhyth m” changes the pronunciation (fails)	-
Number / date test (E4, T4)	Correct expansion	Correct expansion
Listener feedback	Author’s ratings 2–3 / 5; robotic	Understandable, correct pronunciation, unnaturally fast pacing

8.2 Limitations and failures

1. **Overfitting** Both models reach their best validation loss at about 10k steps. The English pitch predictor never improves on its step-1k value.
2. **Pacing and phrasing.** The synthesized speech is faster than the training voice and has pauses only at commas. Both Telugu listeners noticed the pacing
3. **Robotic sound.** The author and the younger listener describe the voices as robotic. Contributing factors that this report can point to, though their individual impact was not measured: phone-level (flat within a phone) pitch and energy, a vocoder that was not fine-tuned on either dataset, and ~4.5 h of training audio.
4. **Front-end failures in English.** The split word in E3 produces a wrong pronunciation. The Telugu word in E4 gets English vowels. The spelt letter “A” is very short.
5. **Sparse English speakers.** The inference speaker has only 6.9 minutes of training audio.
6. **One run per language** within the GPU quota, so no hyper-parameter tuning and no variance estimate across runs.

8.3 Possible improvements, tied to the test sentences

Problem (sentence)	Improvement
One-frame ny, long word-initial vowels (T1, T2a, T2b, T3, T4)	Use AI4Bharat **IndicMFA** pretrained letter level aligners (this needs letter-based input), train MFA on more Telugu data, or learn alignment jointly with the TTS model.
Overfitting (loss curves)	Early stopping at about 10k steps, more data per voice, stronger regularization on the variance predictors, or pre-training on a larger corpus followed by fine-tuning.
Robotic quality (all)	Fine-tune HiFi-GAN on ground-truth-aligned mels of each dataset; try frame-level pitch and energy.
“rhyth m” (E3)	Before G2P, merge a non-word fragment with a following single letter when the joined form is a dictionary word, while keeping the input text unchanged.
“andaroo” (E4)	Add a lexicon entry for common romanized Telugu words, or detect them and route them through the Telugu G2P mapped to the nearest ARPAbet phones.
Short “A” in “A M” (E4)	Add pauses or minimum durations for spelt out letters.
Sparse English speakers	Train on fewer speakers with more data each, or on a single well-covered speaker.

9. Reproducibility

9.1 Software

- Python ≥ 3.10. requirements.txt sets minimum versions: torch ≥ 2.0, numpy ≥ 1.24, scipy ≥ 1.10, librosa ≥ 0.10, soundfile ≥ 0.12, pyworld ≥ 0.3.4, tgt ≥ 1.4.4, g2p_en == 2.1.0, inflect ≥ 5.0, unidecode ≥ 1.3, nltk ≥ 3.8, scikit-learn ≥ 1.2, matplotlib ≥ 3.7, tensorboard ≥ 2.12, pyyaml ≥ 6.0, tqdm ≥ 4.60, pyarrow ≥ 12.0, huggingface_hub ≥ 0.20.
- NLTK data: cmudict, averaged_perceptron_tagger, averaged_perceptron_tagger_eng, punkt.
- Montreal Forced Aligner, installed from conda-forge into its own environment (conda create -n mfa -c conda-forge montreal-forced-aligner python=3.11). Its path is passed via MFA_BIN.
- The runs used a Kaggle notebook with a T4 GPU. The exact package and MFA versions installed on Kaggle at the time were not recorded in the repository.

9.2 Configuration files

- English/config.yaml and Telugu/config.yaml: one file per language with preprocess, model, train and inference sections (dataset, filters, alignment mode, features, model size, optimiser, steps, checkpoint, speaker).
- fastspeech2_core/mfa_train_config.yaml: Telugu MFA training recipe.
- fastspeech2_core/hifigan/config.json: vocoder.
- Environment variables: HF_TOKEN (Svarah is gated), MFA_BIN, and optionally FS2_DATA_ROOT for audio and features (~2 GB per language).

9.3 Commands

Option A – free Kaggle GPU (what was used): import extras/notebooks/kaggle_fastspeech2.ipynb, choose a T4 GPU, turn Internet on, add the HF_TOKEN secret, and set LANG and REPO_URL. The notebook installs MFA, preprocesses, runs a 20-step smoke test, trains for 7 h, plots, synthesizes and zips the outputs. See extras/HOW_TO_RUN_ON_KAGGLE.md.

Option B – any Linux machine with a GPU (replace English with Telugu for the second model):

```
pip install -r requirements.txt
conda create -n mfa -c conda-forge montreal-forced-aligner python=3.11
export MFA_BIN=$(conda run -n mfa which mfa)
export HF_TOKEN=hf_...

# prepare, lexicon, align, features
python English/preprocess.py --config English/config.yaml --stage all
python tools/collect_meta.py --lang English
python English/train.py --config English/config.yaml --max_hours 7
python tools/plot_losses.py --lang English
python English/inference.py --config English/config.yaml --checkpoint best
python tools/export_checkpoint.py --lang English
```

Inference only, from the repository's checkpoints (no training features needed):

```
python English/inference.py \
   --checkpoint English/checkpoints/fastspeech2_english_31000_fp16.pth.tar
python Telugu/inference.py \
   --checkpoint Telugu/checkpoints/fastspeech2_telugu_11000_fp16.pth.tar
python English/inference.py --duration_control 1.2 --text "Slower speech." --id demo
```